



EduConnect

A University ERP + LMS platform for syllabus-based learning, teacher interaction, subscriptions, quizzes, live classes, notes, videos, and academic workflow management.

React + TypeScript

Node.js API

MySQL

Role Based Access Control

ERP + LMS

Prepared for interview explanation, project viva, and company discussion.

1. Project Overview

EduConnect is a centralized academic resource platform for university students. The main problem it solves is scattered learning material: students often search across WhatsApp groups, Telegram channels, random PDFs, and YouTube playlists. EduConnect organizes everything by university syllabus hierarchy.

University -> Course -> Branch/Program -> Scheme -> Semester -> Subject -> Unit -> Topic
-> Notes / Videos / PYQs / Quizzes / Live Classes

The project also includes ERP-style control for admins: academic hierarchy management, teacher approval, subject-wise teacher assignment, content approval, subscription verification, audit logs, and maintenance request workflow.

2. Main Roles

Student
Accesses syllabus-based learning resources, notes, videos, PYQs, quizzes, live classes, progress, bookmarks, subscriptions, and teacher doubt chat.

Teacher
Uploads academic content only for assigned subjects, creates quizzes, uploads PYQs/solutions, schedules live classes, and tracks quiz performance.

Admin
Controls academic hierarchy, users, teacher approval, teacher assignment, content approvals, subscriptions, delete requests, and analytics.

Maintenance
Suggests academic hierarchy changes and sends requests to admin. Maintenance does not directly publish critical changes.

3. Technology Stack

Layer	Technologies / Purpose
Frontend	React 18, TypeScript, Vite, React Router DOM
UI	Tailwind CSS, shadcn/Radix UI components, Lucide React icons, Framer Motion

Layer	Technologies / Purpose
State/Data Fetching	TanStack React Query, custom MySQL client wrapper
Backend	Node.js API using native HTTP style/server module
Database	MySQL 8 with mysql2 driver
Auth/RBAC	Custom app_users + profiles + user_roles tables
File Handling	Local upload API and public upload paths for notes/payment screenshots/PYQs
Charts/Analytics	Recharts for dashboard/analytics visualization
Dev Tools	npm scripts: npm run api, npm run dev, npm run build

4. Modules

Module	Explanation
Authentication & RBAC	Signup/login, profile creation, role selection, role protected routes for student, teacher, admin, and maintenance.
Student Dashboard	Shows enrolled/available learning flow, subjects, teachers, quizzes, subscription access, live classes, and activity entry points.
Syllabus LMS	University -> Course -> Branch -> Scheme -> Semester -> Subject -> Unit -> Topic drill-down with notes/videos/PYQs.
Smart Notes	Topic-mapped notes with readable same-page content and PDF download/save option.
Video Learning	Topic-wise video player workflow inspired by modern LMS navigation.
Quiz Arena	Subject-wise quizzes, 20 free questions, premium full bank, 20 sec/question timer, score, correction, leaderboard.
Previous Year Papers	Year-wise PYQ access, subject mapping, optional teacher solutions.
Subscription & Payment	Student selects semester plan, scans QR, uploads screenshot/transaction ID, admin verifies and activates premium.
Teacher Dashboard	Teacher sees assigned subjects, uploads notes/videos/PYQs, creates quizzes, schedules/starts live classes.
Teacher Assignment	Admin approves teachers and maps teachers subject-wise so teachers access only assigned subjects.
Admin Dashboard	ERP control center for hierarchy, teachers/students, approvals, subscriptions, analytics, audit logs.
Maintenance Dashboard	Maintenance team proposes hierarchy/content structure changes; admin approves/rejects before publishing.
Approval Center	Admin reviews uploaded content, delete requests, hierarchy requests, and subscription payments.
Doubt Chat	Student can start chat thread with teacher; teacher can reply in discussion workflow.

5. Role Based Access Control

RBAC is implemented using `user_roles`. Routes are protected using a React role guard. Admin has broad access. Teacher, student, and maintenance users are limited to their respective dashboards and actions.

- **Admin:** Full control and approval authority.
- **Teacher:** Can manage content/quizzes/live classes for assigned subjects.
- **Student:** Can consume resources and request subscription access.
- **Maintenance:** Can create hierarchy update requests for admin review.

6. Academic Hierarchy Workflow

1. Admin creates university, course, branch/program, scheme, semester subjects, units, and topics.
2. Admin approves teacher accounts.
3. Admin assigns teachers to specific subjects.
4. Teacher uploads notes, videos, PYQs, quizzes, and live class links for assigned subjects.
5. Admin reviews/approves content where approval workflow is required.
6. Student opens syllabus and consumes resources topic-wise.

7. Student Learning Workflow

Login -> Dashboard -> Syllabus -> Select University/Course/Semester -> Subject -> Unit -> Topic -> Notes / Video / PYQ / Quiz

Students can view free preview content. Premium semester subscription unlocks full subject resources, full quiz question bank, downloadable notes, videos, and PYQs.

8. Subscription Workflow

1. Student selects semester premium plan.
2. Student scans PhonePe/UPI QR and pays.
3. Student uploads payment screenshot and transaction ID.
4. System creates `subscriptions` and `payments` records with pending status.
5. Admin reviews screenshot and activates/rejects subscription.
6. After approval, student receives premium access for that semester.

9. Quiz Workflow

- Teacher/admin creates quiz, questions, and options.
- Student starts quiz from Quiz Arena.
- Free students access up to 20 questions per subject quiz.
- Premium students access the full quiz bank where available.
- Timer is 20 seconds per question.
- On submit/auto-submit, system stores one `quiz_attempts` row and multiple `student_answers` rows.
- Leaderboard is generated from completed quiz attempts.

10. Database Tables

Table	Purpose	Important Columns
app_users	Authentication account records: login email and password hash.	id, email, password, created_at
profiles	User profile details used across student, teacher, admin, and maintenance views.	user_id, full_name, phone, gender, university_id, course_id, branch_id, semester
user_roles	Role based access control mapping.	user_id, role
universities	Top level academic institution records.	name, short_name
courses	Courses under a university, such as MCA or B.Tech.	university_id, name, total_semesters
branches	Program or specialization under a course.	course_id, name, code
schemes	Curriculum version or academic scheme.	branch_id, name, year, is_active
subjects	Semester subjects mapped to a scheme.	scheme_id, name, code, semester, is_free
units	Subject units.	subject_id, unit_number, name
syllabus_topics	Topic-wise syllabus index under each unit.	unit_id, title, topic_order, description
content	Teacher uploaded notes, PDFs, and video links mapped to subject/unit/topic.	subject_id, unit_id, syllabus_topic_id, content_type, file_url, video_url, approval_status
content_topics	Optional internal topic/page indexing for uploaded notes.	content_id, title, page_number, topic_order
content_views	Tracks student content views.	user_id, content_id, viewed_at
bookmarks	Student saved content items.	user_id, content_id
previous_papers	PYQ records with optional teacher solution text/file.	subject_id, unit_id, year, file_url, solution_text

Table	Purpose	Important Columns
course_enrollments	Student enrollment/progress per subject.	user_id, subject_id, progress
teacher_assignments	Admin subject-wise teacher access mapping.	teacher_id, subject_id, assigned_by, is_active
live_sessions	Teacher live classes and Meet/session links.	subject_id, title, meet_link, is_live, started_at, ended_at
quizzes	Teacher/admin created quiz containers.	subject_id, title, difficulty_level, is_published, time_limit_minutes
questions	Questions under each quiz.	quiz_id, question_text, question_order, marks
options	MCQ options for questions.	question_id, option_text, is_correct, option_order
quiz_attempts	One quiz attempt per student/quiz attempt.	quiz_id, user_id, score, total_questions, total_marks, is_completed
student_answers	Individual answer records for each submitted attempt.	attempt_id, question_id, selected_option_id, is_correct
quiz_daily_usage	Quiz usage/quota tracking support.	user_id, usage_date, attempts_count, questions_attempted
subscriptions	Semester premium subscription requests and active plans.	user_id, semester, amount, status, plan_type, activated_by
payments	Payment proof, transaction and admin verification records.	user_id, subscription_id, amount, transaction_id, payment_screenshot_url, verified_by
hierarchy_requests	Maintenance requests for academic hierarchy changes.	target_table, action_type, payload, status, requested_by, reviewed_by
delete_requests	Admin approval workflow before deleting sensitive records.	table_name, record_id, reason, status
audit_logs	System activity logs for create/update/delete operations.	table_name, record_id, action_type, old_data, new_data, actor_id
chat_threads	Student-teacher doubt discussion thread headers.	student_id, teacher_id, subject, status
chat_messages	Messages inside doubt discussion threads.	thread_id, sender_id, message
teacher	Legacy/simple teacher table still present in DB.	teacher_id, name, Email_id

11. Important Relationships

- app_users.id -> profiles.user_id and user_roles.user_id
- universities.id -> courses.university_id
- courses.id -> branches.course_id
- branches.id -> schemes.branch_id
- schemes.id -> subjects.scheme_id

- `subjects.id` -> `units.subject_id`, `content.subject_id`, `quizzes.subject_id`, `previous_papers.subject_id`
- `units.id` -> `syllabus_topics.unit_id`
- `teacher_assignments.teacher_id` links teacher user to `teacher_assignments.subject_id`
- `quizzes.id` -> `questions.quiz_id` -> `options.question_id`
- `quiz_attempts.id` -> `student_answers.attempt_id`
- `subscriptions.id` -> `payments.subscription_id`

12. API / Backend Summary

The backend is a Node.js MySQL API. The frontend uses a custom `mysqlClient` wrapper that provides Supabase-like calls such as `from(table).select()`, `insert()`, `update()`, `delete()`, and `rpc()`.

- `npm run api` starts the MySQL API server.
- `npm run dev` starts the Vite frontend.
- API base URL is configured by `VITE_API_BASE_URL`.
- Authentication uses local session storage and DB tables.

13. Interview Explanation Script

Short version: EduConnect is a syllabus-based LMS for university students with ERP-style admin control. It organizes notes, videos, PYQs, quizzes, live classes, subscriptions, and teacher interaction according to the academic hierarchy.

Detailed version: I built EduConnect using React, TypeScript, Tailwind CSS, Node.js, and MySQL. The system uses role based access control. Students access structured learning resources. Teachers upload content only for subjects assigned by admin. Admin manages hierarchy, teacher assignments, content approvals, subscriptions, and analytics. Maintenance users can raise academic update requests that admin approves before publishing.

14. Strengths Of The Project

- Solves a real student problem: scattered academic content.
- Uses structured syllabus hierarchy instead of random resources.
- Includes real RBAC and subject-wise teacher access.
- Has subscription/payment verification workflow with screenshot proof.
- Stores quiz attempts and student answers in normalized DB tables.
- Includes admin/teacher/student/maintenance workflows, making it more than a simple LMS.

15. Possible Future Improvements

- JWT based authentication and password reset email integration.
- Cloud file storage such as S3/Cloudinary for PDFs and screenshots.
- More detailed analytics for student progress and teacher performance.
- AI based topic explanation and automatic note summarization.
- Payment gateway integration instead of manual screenshot verification.
- Better notification system for live classes and approvals.

16. Commands To Run

- `npm install` - install dependencies.
- `npm run api` - start backend API.
- `npm run dev` - start frontend.
- `npm run build` - production build verification.